

Justificación del diagrama de clases

Sistema de gestión de prendas de vestir

1. El problema que se está modelando

El sistema busca representar y gestionar distintos tipos de prendas de vestir (camisas, pantalones y calcetas). De cualquier prenda en general necesitamos conocer características básicas compartidas, como su talla y color. Además, todas las prendas comparten acciones comunes: una persona puede ponérselas, quitárselas o lavarlas.

Sin embargo, cada tipo específico de prenda tiene características únicas (por ejemplo, una camisa tiene un tipo de manga, pero un pantalón tiene un tipo de corte). Asimismo, aunque todas las prendas se lavan, el proceso de lavado puede variar dependiendo del tipo de prenda.

Este diagrama de clases representa exactamente esa estructura: cómo se organizan jerárquicamente las prendas, reutilizando atributos comunes y definiendo comportamientos específicos mediante la herencia.

2. Por qué se eligió un diagrama de clases

A diferencia de otros modelos, el diagrama de clases muestra la estructura de los datos (qué atributos tiene cada entidad). Es la herramienta correcta aquí porque el problema describe entidades estáticas, sus características (talla, color) y sus capacidades (métodos), en lugar del orden en que ocurren los eventos. Nos permite visualizar conceptos clave de la programación orientada a objetos como la herencia, el encapsulamiento y el polimorfismo.

3. Los participantes: por qué son estos y no otros

- **Prenda (Clase Padre / Superclase):** Se modela como una entidad central y abstracta porque agrupa todos los datos y comportamientos comunes. No tendría sentido repetir los atributos "talla" y "color" en cada prenda individual; centralizarlos aquí optimiza el modelo.
- **Camisa, Pantalón, Calcetas (Clases Hijas / Subclases):** Se modelan como clases separadas porque representan objetos concretos que tienen propiedades

exclusivas. Una camisa necesita registrar su tipoDeManga, un pantalón su tipoDeCorte y las calcetas su longitud.

4. Estructura y justificación de métodos y atributos

La siguiente tabla explica de dónde sale cada elemento del diagrama y por qué tiene esa visibilidad y ubicación.

Elemento (Atributo/Método)	Justificación (por qué existe y su diseño)	Notación usada
talla, color (en Prenda)	Son características universales de cualquier ropa. Se colocan en la superclase para que las subclases los hereden automáticamente sin reescribir código.	Atributos protegidos (#). Esto permite que las clases hijas puedan acceder a ellos directamente, pero quedan ocultos para el resto del sistema.
ponerse(), quitarse() (en Prenda)	Son acciones genéricas que aplican por igual a cualquier ropa. Su implementación no cambia, por lo que se definen de forma concreta en el padre.	Métodos públicos (+), indicando que cualquier actor o parte del sistema puede invocar estas acciones.
lavar() (en Prenda)	Todas las prendas se ensucian y deben lavarse, por lo que es obligatorio que exista. Sin embargo, no se puede definir <i>cómo</i> lavarla de forma genérica (una camisa de seda se lava distinto a unos calcetines de algodón).	Método abstracto (<<abstract>> y/o cursiva) y público (+). Obliga a las clases hijas a programar su propia versión del lavado.
tipoDeManga,	Son atributos exclusivos. Una	Atributos privados (-).

tipoDeCorte, longitud	camisa no tiene "longitud" en el mismo sentido que las calcetas, ni las calcetas tienen "tipo de corte".	Refuerzan el encapsulamiento; nadie externo puede modificarlos directamente.
Camisa(...), Pantalon(...), Calcetas(...)	Son los métodos constructores. Sirven para crear (instanciar) los objetos cuando el sistema arranca, recibiendo los datos necesarios (talla, color y el atributo específico).	Métodos constructores públicos (+).
getTipoDeManga(), etc.	Al ser los atributos específicos privados (-), el sistema necesita una forma segura de consultarlos o leerlos sin alterarlos.	Métodos "getter" públicos (+) que devuelven un tipo String.
lavar() (en subclases)	Es la respuesta obligatoria al método abstracto del padre. Cada prenda provee su propia instrucción específica de lavado (polimorfismo).	Métodos públicos (+).

5. Qué casos cubre el diagrama y cuáles no (y por qué)

- **Casos que sí están cubiertos:**
 - La reutilización de código mediante la herencia (los atributos comunes no se repiten).
 - El encapsulamiento correcto de los datos mediante visibilidad (+, -, #).
 - La obligación de implementar comportamientos específicos a través del polimorfismo (método lavar()).
- **Casos que NO están cubiertos (y por qué es correcto que no lo estén):**

- **El flujo temporal o de ejecución:** Este diagrama no dice si la prenda se lava antes o después de ponérsela; eso corresponde a un diagrama de secuencia o de actividad.
- **Quién es el dueño de la prenda:** No se modela la clase Persona o Armario. El enfoque está estrictamente en la jerarquía de las prendas, lo cual es válido si el ejercicio trata de herencia básica.
- **Validaciones de datos:** No se detalla qué pasa si se ingresa una talla negativa o un color que no existe.

6. Cómo leer la notación (para explicar los símbolos si preguntan)

- **Rectángulo dividido en 3 partes = Clase:** La parte superior es el nombre, la de en medio los atributos (variables) y la inferior los métodos (funciones).
- **<<Abstract>> (Estereotipo):** Indica que de esa clase no se pueden crear objetos directamente (no puedes instanciar una "Prenda" genérica en el código, tiene que ser forzosamente una Camisa, Pantalón, etc.).
- **Signo - = Privado (Private):** Solo la propia clase puede ver y usar ese elemento.
- **Signo # = Protegido (Protected):** La clase y sus "hijas" pueden usar ese elemento.
- **Signo + = Público (Public):** Cualquier parte del sistema puede ver y usar ese elemento.
- **Línea continua con flecha triangular vacía:** Es el símbolo de Generalización (Herencia). Se lee de abajo hacia arriba: "Una Camisa es *una* Prenda". Apunta siempre a la superclase.

7. Posibles preguntas del profesor y cómo responderlas

- **P: ¿Por qué la clase Prenda es abstracta?**
 - **R:** Porque "Prenda" es solo un concepto para agrupar características. En el mundo real (y en nuestro sistema), tú no te pones una "prenda" abstracta, te pones un objeto físico concreto: una camisa o un pantalón. Por tanto, el sistema no debe permitir instanciar objetos directos de la clase Prenda.
- **P: ¿Por qué usaste visibilidad protegida (#) en talla y color, pero privada (-) en tipoDeManga?**

- **R:** Los atributos comunes se dejaron protegidos para que las clases hijas (Camisa, Pantalón) tengan acceso directo a ellos por herencia sin necesidad de crear métodos 'getters' y 'setters' en el padre solo para la comunicación interna. Sin embargo, los atributos propios de cada subclase son privados para cumplir con el principio de encapsulamiento hacia el resto del sistema, exponiéndolos solo mediante sus métodos 'get'.
- **P: ¿Por qué lavar() está en la clase padre y también en las clases hijas?**
 - **R:** Porque aplica el concepto de Polimorfismo. En el padre se define como método abstracto para *obligar* por contrato a que toda prenda pueda lavarse. Pero como cada tela o prenda tiene instrucciones distintas de lavado, cada clase hija debe sobrescribir el método e implementar su propia lógica.
- **P: ¿Qué significa el triángulo blanco en las líneas que conectan las cajas?**
 - **R:** Representa la herencia. Indica que las clases inferiores heredan todos los atributos y métodos no privados de la clase a la que apunta el triángulo.



